CPIA · DOCUMENTACIÓN TÉCNICA DE CÓDIGO

Explicación Línea a Línea del Script

Desglose completo y justificación de `problemas/5_Materials_Uriel/materiales_uriel.py`

Estudiante: Uriel | **Algoritmo:** `sklearn.cluster.DBSCAN` (ε = 0.65, min_samples = 3) | **Archivo explicado:** `materiales_uriel.py`

1. Bloque de Importaciones (Líneas 13 – 18)

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
```

- **Línea 13 (`Path`):** Gestión de rutas multiplataforma (evita incompatibilidades de barras entre Windows y Linux/Mac).
- **Línea 14 (`numpy`):** Manipulación de matrices y operaciones vectorizadas, incluyendo el cálculo manual de la media y la desviación típica para el escalado.
- **Línea 15 (`pandas`):** Carga del CSV en DataFrame y cálculo de matrices de contingencia cruzada.
- **Línea 16 (`matplotlib.pyplot`):** Motor gráfico para la visualización 2D de clusters y exportación de imágenes.
- **Líneas 17 – 18 (`scikit-learn`):** Importa directamente la clase oficial `DBSCAN` y la métrica de validación `silhouette_score` de la librería de Machine Learning de referencia.

2. Carga del Dataset y Rutas Relativas (Líneas 23 – 30)

```
DATOS_DIR = Path(__file__).resolve().parents[2] / "datos"
CSV_PATH = DATOS_DIR / "materiales_electricos.csv"

if not CSV_PATH.exists():
    raise FileNotFoundError(f"No se encontró el archivo de datos en: {CSV_PATH}")

df = pd.read_csv(CSV_PATH)
print(f"Dataset cargado con éxito ({len(df)} materiales).")
```

- **Línea 23:** `Path(__file__).resolve().parents[2] / "datos"` accede a la raíz del repositorio y entra en `datos/` . Al estar el script dentro de `problemas/5_Materials_Uriel/` , el índice `parents[2]` sube exactamente los dos niveles requeridos hasta la raíz.
- **Líneas 26 – 27:** Control de errores: si el CSV no existe, avisa con claridad al usuario.
- **Línea 29:** Carga los 57 materiales reales en el DataFrame `df` .

3. Selección de Características y Estandarización Manual Z-Score (Líneas 37 – 44)

```
features = ["log_rho_300K", "alpha_coef"]
X_raw = df[features].values

# Estandarización manual: Z = (X - μ) / σ
media = np.mean(X_raw, axis=0)
desv = np.std(X_raw, axis=0)
X_scaled = (X_raw - media) / desv
```

- **Líneas 37 – 38:** Extrae la matriz numérica con las dos dimensiones físicas: la resistividad logarítmica (rango ≈ 31 órdenes de magnitud) y el coeficiente térmico α (rango ≈ 0.08).
- **Líneas 42 – 44 (Estandarización Manual):** Se calcula explícitamente la media **μ** con `np.mean(X_raw, axis=0)` y la desviación típica **σ** con `np.std(X_raw, axis=0)` para cada columna, aplicando la fórmula de estandarización **Z = (X − μ) / σ**. Esto asegura que ambas variables tengan media 0 y varianza 1, evitando que la escala de resistividad opaque la variación del coeficiente térmico en el cálculo de distancias de DBSCAN.

4. Aplicación de DBSCAN de scikit-learn (Líneas 49 – 53)

```
EPS = 0.65
MIN_SAMPLES = 3

dbscan = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES)
df["cluster"] = dbscan.fit_predict(X_scaled)
```

- **Líneas 49 – 50:** Fija los dos hiperparámetros de DBSCAN: el radio de vecindad `eps = 0.65` y el umbral de densidad `min_samples = 3`.
- **Línea 52:** Instancia el modelo `DBSCAN` de scikit-learn.
- **Línea 53:** `.fit_predict(X_scaled)` ejecuta el algoritmo en un solo paso:
  1. Construye el árbol espacial de vecinos de cada punto.
  2. Identifica los puntos núcleo (los que tienen ≥ 3 vecinos dentro de un radio euclídeo de 0.65).
  3. Expande los clusters encadenando puntos núcleo y frontera mediante conectividad por densidad.
  4. Asigna etiquetas enteras (0, 1, 2...) y marca con `-1` cualquier punto aislado (ruido).

5. Evaluación con Silhouette Score de scikit-learn (Líneas 58 – 62)

```
mask_validos = df["cluster"] != -1
if len(set(df["cluster"][mask_validos])) > 1:
    score_sil = silhouette_score(X_scaled[mask_validos], df["cluster"][mask_validos])
else:
    score_sil = None
```

- **Línea 58:** `mask_validos` filtra los puntos que no son ruido (`cluster != -1`). En DBSCAN, el ruido no forma un cluster cohesionado, por lo que la literatura científica evalúa la silueta sobre los núcleos densos reconocidos.
- **Línea 60:** Llama a `silhouette_score()` de scikit-learn, obteniendo una puntuación de **0.7858**, lo que certifica matemáticamente una separación nítida entre conductores, semiconductores y aislantes.

6. Métricas y Matriz de Contingencia (Líneas 67 – 86)

```
clusters_encontrados = sorted(set(df["cluster"]) - {-1})
n_clusters = len(clusters_encontrados)
n_ruido = int(np.sum(df["cluster"] == -1))

print(f"Clusters descubiertos por densidad : {n_clusters}")
print(f"Silhouette Score (núcleos densos) : {score_sil:.4f}")

print("\nMatriz de contingencia (Clase real vs. Cluster DBSCAN):")
print(pd.crosstab(df["clase_real"], df["cluster"], margins=True))
```

- **Líneas 67 – 69:** Calcula el número de clusters reales descubiertos y la cantidad de puntos de ruido.
- **Línea 80:** `pd.crosstab(df["clase_real"], df["cluster"])` genera la matriz de contingencia que valida el clustering no supervisado frente a la física real: los 35 metales se asignan íntegramente al Cluster 0, los 10 aislantes al Cluster 1 y los semiconductores tecnológicos al Cluster 2.

7. Visualización Científica con Matplotlib (Líneas 91 – 161)

```
plt.figure(figsize=(10, 6.5))

paleta_colores = {
    0: ("#d97706", "Cluster 0 (Conductores)"),
    1: ("#2563eb", "Cluster 1 (Aislantes)"),
    2: ("#059669", "Cluster 2 (Semiconductores)"),
}

for cl in clusters_encontrados:
    sub = df[df["cluster"] == cl]
    color, nombre_cl = paleta_colores.get(cl, ("#6b7280", f"Cluster {cl}"))
    plt.scatter(sub["log_rho_300K"], sub["alpha_coef"], c=color, s=70, alpha=0.85, ...)

plt.axhline(0, color="gray", linestyle="--", linewidth=1, alpha=0.7, label=r"Frontera $\alpha = 0$")

# Anotar nombres de materiales destacados en el gráfico
materiales_destacados = ["copper", "silver", "silicon", "germanium", "gaas", "glass", "ptfe", "quartz"]
for _, r in df.iterrows():
    if any(k in r["material"].lower() for k in materiales_destacados):
        plt.annotate(r["material"].split("(")[0].strip(), (r["log_rho_300K"], r["alpha_coef"]), ...)

FIGURA_PATH = Path(__file__).resolve().parent / "clusters_materiales_dbscan.png"
plt.savefig(FIGURA_PATH, dpi=300)
plt.show()
```

- **Líneas 96 – 100:** Asigna una paleta cromática semántica: ámbar para metales, azul para aislantes y verde esmeralda para semiconductores.
- **Línea 116:** `plt.axhline(0, ...)` dibuja la línea discontinua de $\alpha = 0$, fundamental para explicar visualmente que los metales tienen pendiente térmica positiva ($\alpha > 0$) y los semiconductores negativa ($\alpha < 0$).
- **Líneas 119 – 133:** Identifica materiales emblemáticos (Cobre, Silicio, Vidrio, Teflón, etc.) y añade etiquetas de texto automáticas con `plt.annotate`.
- **Línea 143:** Guarda la figura resultante con resolución de 300 DPI en `problemas/5_Materials_Uriel/clusters_materiales_dbscan.png` para incrustarla en diapositivas o informes.